MontuDB 4.8 Manual

Reference manual for **MontuDB 4.8** — a fork of PostgreSQL 17.2 that adds the opt-in *Montu Optimizer*. It covers what MontuDB is, the optimizer modules, the hint and GUC activation surface, the possible combinations, and how to verify what fired. Source of truth: the parser `src/backend/optimizer/montu/montu_hint.c` and the GUCs registered in the `montu_*` modules.

---

1. Overview — what is MontuDB 4.8?

MontuDB 4.8 is a fork of PostgreSQL 17.2 that adds a single extra component on top of the stock database: the *Montu Optimizer*. Everything else — the storage engine, MVCC, WAL, the executor, the wire protocol, the on-disk page format — is unmodified PostgreSQL 17.2. `server_version_num` remains `170002`, and the same data directory can be served interchangeably by a stock PostgreSQL 17.2 binary or by MontuDB 4.8.

The Montu Optimizer is an **opt-in planning layer** that sits alongside PostgreSQL's cost-based planner and intervenes only when you ask it to. It exists to address two well-known limitations of the stock planner on heavy analytical / reporting workloads:

- **Bad estimates → bad plans.** PostgreSQL chooses a plan from *estimated* row counts and costs. On large queries those estimates drift far from reality — a filter the planner thinks returns 10 rows actually returns 10,000; a join's real fan-out is nothing like the estimate — and the resulting plan is slow. The Montu Optimizer's **RealCost** engine runs bounded *probes* to measure the *real* cardinalities, then picks the plan by a **dual-plan comparison** (real vs estimated) instead of trusting the estimate blindly.
- **Structural rewrites stock never attempts.** Some rewrites are highly effective for reporting but are simply not in the stock planner's repertoire: collapsing a self-join aggregation into a **window function** (OLAP), or materializing a repeated sub-tree **once** in a shared CTE instead of recomputing it for every `UNION` arm (Condense). MontuDB detects these patterns, costs them, and adopts them only when they win.

A fourth module, **MRRV**, validates the *risk* of a heuristic rewrite before it is adopted. The four modules are detailed in Section 3.

The entire design is built around **not surprising you**:

Principle	What it means
Default-OFF / stock parity	With no hint and no `montu.*` GUC changed, behavior is **bit-identical to stock PostgreSQL 17.2** — same results, same plans. You only pay for / gain anything when you explicitly enable it.
Fail-open	A malformed hint is ignored and the query runs as stock; the hint never raises an error (unless you opt into strict mode).
Bounded effort	Probing runs under a time / effort budget (`budget_ms`, `total_pcu`) — it never runs unbounded.
Conservative adoption	Structural rewrites are adopted **only when validated and provably cheaper** (default margin ≥15%).
Narrow scope	The optimizer engages on `SELECT` only, under `READ COMMITTED`. DML/DDL and other isolation levels always run stock.
Full compatibility	On-disk format and protocol are unchanged → a stock cluster and MontuDB 4.8 are interchangeable on the same datadir.

Two ways to turn it on:

1. **SQL hint** — `montu_optimizer(...)`, applied per query (Section 2).
2. **GUCs** — `SET montu.*`, applied per session/transaction (Section 6).

A single hint fires **one module at a time**. **Composition** — running more than one structural optimizer together, or letting them fire in nested query levels — is done **via GUC only** (Section 4).

Verifying it: `EXPLAIN` prints a marker when the optimizer engages — e.g. `Montu Optimizer: status=engaged-skeleton level=3`, a `Montu RealCost Budget: …` line, and `Montu OLAP:` / `Montu Condense:` sections when those modules evaluate candidates (Section 7).

2. The base hint — `montu_optimizer(N)`

```
-- Canonical form (recommended): right AFTER the SELECT
SELECT /*+ montu_optimizer(5) */ col, ...
FROM   ...

-- Legacy form (still accepted): comment at the START of the query
/*+montu_optimizer(5)*/ SELECT col, ... FROM ...
```

Rule	Detail
Position	Canonical = after the `SELECT`. The leading form still works, but ORMs/BI tools/proxies tend to strip comments from the start — that's why the after-`SELECT` form is recommended.
Case	Case-insensitive: `montu_optimizer`, `MONTU_OPTIMIZER`, `Montu_Optimizer`.
Whitespace	Tolerates spaces and line breaks: `/*+ montu_optimizer ( 4 ) */`.
N	**Analysis/probing budget** (effort). `0` = explicit OFF; `1..5` = ON, increasing; `>5` → **clamps to 5**. In EXPLAIN, `N` maps to `total_pcu` (probe cost units: ~9000 at N=3, ~25000 at N=5) under a time ceiling `budget_ms` (~60s). It is not "N minutes".
Invalid	`montu_optimizer(abc)`, `(-1)`, `(3.5)` → **malformed** ⇒ hint ignored (fail-open: the query runs normally, like stock).
Multiple	More than one directive in the same block ⇒ **the last one wins** (does not stack).

`N` alone (≥1) triggers **RealCost** (Section 3.1).

3. The modules and their objectives

Module	Triggered via	Objective (what it's for)
RealCost	`montu_optimizer(N)`, N≥1	Fix plans when the **planner's cardinality/cost estimates are wrong**.
OLAP	`olap=>MODE` or `SET montu.olap_mode`	Convert **auto-join/self-join → window function** (reporting queries).
MRRV (rewrite)	`rewrite=>MODE`	**Validate the risk** of a heuristic rewrite before adopting it.
Condense	`condense=>MODE` or `SET montu.condense`	**Materialize once** repeated blocks/tables (e.g., UNION arms).

3.1 RealCost — `montu_optimizer(N)` (N = 1..5)

Objective: long / reporting queries where the cost estimated by the planner diverges greatly from the real one. Depending on the budget `N` (higher `N` = more *probing* allowed), Montu runs validation *probes* to **measure the real cardinality** and picks the plan via **dual-plan comparison** (real vs estimated), respecting a time ceiling (`budget_ms`). Classic examples: a filter estimated at 10 rows that in practice reads 10,000; a join whose real fan-out is very different from the estimated one.

RealCost modifiers: - `,CACHE` — reuses the RealCost decision on subsequent executions. E.g.: `montu_optimizer(3,CACHE)`. - `,INJECT,'<json>'` — injects external estimates (used with `N=0`). E.g.: `montu_optimizer(0,INJECT,'{...}')`.

3.2 OLAP — `olap=>MODE`

Objective: detect the *auto-join* pattern (a table joined with itself for aggregation) and **collapse it into a window function**, which is usually much cheaper in reports.

Value	Effect

`off`	Disabled (stock planner).
`always`	Converts **ignoring cost**.
`validate`	Converts **only if** validated and provably cheaper.
`validate_dryrun`	Validates + costs, but **never adopts** (diagnostic).

Equivalent session GUC: `montu.olap_mode` (same values). Adoption margin: `montu.olap_cost_margin` (default `0.15` ⇒ it must be ≥15% cheaper).

3.3 MRRV — `rewrite=>MODE`

Objective: risk validation of heuristic *rewrites* — before applying a rewrite, MRRV measures via probes whether it is **safe and beneficial**.

Value	Effect
`off`	Silent (absent).
`log`	Only logs what it would do.
`on`	Validates and adopts when it wins.
`strict`	Strict mode.

It is **hint-only** (there is no mode GUC). There is a master *kill-switch*: `montu.rewrite_enable` (default `on`; with `off` the hint is ignored with a WARNING).

3.4 Condense — `condense=>MODE`

Objective: when the same table/block appears multiple times in the query (e.g., several arms of a `UNION`, repeated subqueries), Condense **materializes that block once** in a shared hidden CTE and reuses it, adopting **only if** the RealCost dual-plan comparison wins.

Value	Effect
`off` / `false`	Stock (default).
`on` / `true`	Detects + synthesizes + adopts if it wins on cost.
`log`	Runs the whole matrix and costs it, but **never adopts**.
`auto`	Reserved (treated as `off`).

Equivalent GUC: `montu.condense`. Triggers: `montu.condense_min_consumers` (default `2` — minimum number of uses of the block), `montu.condense_cost_margin` (`0.15`), `montu.condense_max_temp_mb` (`512`).

---

4. V4 modes (composition and per-level) — GUC only

The **4.8 = "V4 Compose"** hallmark: the structural optimizers can fire at **any level** and **co-fire** in a single query. This is controlled by GUC (not by the hint):

GUC	Default	Objective
`montu.compose`	`off`	Allows a single plan to carry **OLAP per-level AND the Condense shared CTE** together.
`montu.olap_perlevel`	`off`	Lets OLAP fire **inside a nested subquery** in the FROM (not only at the top).
`montu.condense_perlevel`	`off`	Lets Condense descend into **set-operations / nested CTE bodies**.

All default-OFF ⇒ when off, the behavior is byte-identical to stock.

---

5. Possible combinations

Combination rules:

1. In a **single** `montu_optimizer(...)`: `N` (positional) + comma-separated modifiers.
2. `olap=>`, `rewrite=>`, and `condense=>` are **mutually exclusive with each other** within a directive — via hint, **one structural module per query**.
3. `,CACHE` and `,INJECT` are orthogonal (they belong to RealCost).
4. For **OLAP + Condense together** (compose) or firing at **nested levels** → use **GUCs** (`montu.compose`, `montu.*_perlevel`), not the hint.
5. Multiple directives in the same block **do not add up** — the last one wins.

Recipe table:

I want to…	How
Disable (stock)	nothing, or `montu_optimizer(0)`
RealCost (effort 3)	`SELECT /*+ montu_optimizer(3) */ …`
RealCost + cache	`SELECT /*+ montu_optimizer(3,CACHE) */ …`
Inject estimates	`SELECT /*+ montu_optimizer(0,INJECT,'{…}') */ …`
OLAP in one query	`SELECT /*+ montu_optimizer(0, olap=>validate) */ …`
Condense in one query	`SELECT /*+ montu_optimizer(0, condense=>on) */ …`
MRRV in one query	`SELECT /*+ montu_optimizer(0, rewrite=>on) */ …`
OLAP for the whole session	`SET montu.olap_mode = validate;`
OLAP + Condense (compose)	`SET montu.olap_mode=validate; SET montu.condense=on; SET montu.compose=on;`
Structural in nested subqueries	`SET montu.olap_perlevel=on;` and/or `SET montu.condense_perlevel=on;`

Note: grammatically `N≥1` can coexist with a structural modifier in the same hint (e.g., `montu_optimizer(3, olap=>validate)` = RealCost budget 3 **and** OLAP validate). The **canonical, tested** forms, however, are: `montu_optimizer(N)` for RealCost and `montu_optimizer(0, <module>=>MODE)` for the structural ones. To actually combine multiple structural optimizers, use the compose GUCs above.

---

6. Tuning GUCs (quick reference)

GUC	Default	What it's for
`montu.olap_mode`	`off`	Session OLAP mode (`off`/`always`/`validate`/`validate_dryrun`).
`montu.condense`	`off`	Session Condense mode (`off`/`on`/`log`/`auto`).
`montu.rewrite_enable`	`on`	MRRV master kill-switch.
`montu.compose`	`off`	OLAP per-level + Condense in a single plan.
`montu.olap_perlevel` / `montu.condense_perlevel`	`off`	Firing at nested levels.
`montu.*_cost_margin` (olap/condense/rewrite)	`0.15`	Minimum cost gain to adopt (15%).
`montu.scheduler_mode`	`adaptive`	RealCost scheduler (`fixed`/`adaptive`/`exhaustive`).
`montu.olap_strict_hint`	`off`	Invalid hint value (`olap`/`rewrite`/`condense`) becomes **ERROR** (vs WARNING).
`montu.condense_min_consumers`	`2`	Minimum number of uses of a block to become a candidate.

(There are dozens of finer-grained `montu.*` GUCs; see `SELECT name, short_desc FROM pg_settings WHERE name LIKE 'montu.%';`.)

---

7. Verifying what fired (EXPLAIN)

```
EXPLAIN (COSTS off) SELECT /*+ montu_optimizer(3) */ 1;
--  Montu Optimizer: status=engaged-skeleton level=3
--  Montu RealCost Budget: mode=adaptive budget_ms=60000 total_pcu=9000 ... stop_reason=not_set

EXPLAIN SELECT /*+ montu_optimizer(0, olap=>validate) */ ... ;
--  ... "Montu OLAP:" section with the evaluated candidates

EXPLAIN SELECT /*+ montu_optimizer(0, condense=>on) */ ... ;
--  ... "Montu Condense:" section with the shared blocks
```

- If the hint is **absent/disabled/malformed**, no marker appears (or an off `status` appears) — and the plan is the stock PostgreSQL one.
- `N>5` appears as `level=5` (clamp).

---

8. Practical examples

```
-- 1) Heavy report: let RealCost spend up to 5 "levels" measuring real cardinalities
SELECT /*+ montu_optimizer(5) */
       cliente_id, sum(valor)
FROM   pedidos p JOIN itens i USING (pedido_id)
WHERE  p.data >= date '2026-01-01'
GROUP  BY cliente_id;

-- 2) Window self-join: try the OLAP form only if it is provably cheaper
SELECT /*+ montu_optimizer(0, olap=>validate) */
       t.id, t.v - lag(t.v) OVER (ORDER BY t.id)
FROM   serie t;

-- 3) UNION with arms that repeat the same sub-tree: materialize once
SELECT /*+ montu_optimizer(0, condense=>on) */ ...
UNION ALL
SELECT ...;

-- 4) Reporting session: OLAP + Condense composing, including in subqueries
SET montu.olap_mode = validate;
SET montu.condense  = on;
SET montu.compose   = on;
SET montu.olap_perlevel = on;
SET montu.condense_perlevel = on;
-- ... run your reporting queries normally ...
RESET ALL;  -- back to stock
```

---

9. Safety / semantics

- **Fail-open:** a malformed hint ⇒ ignored, the query executes like stock (it never raises an error because of the hint, except for an invalid value with `montu.olap_strict_hint=on`).
- **Default-OFF / parity:** with no hint and no GUC, the result and the plan are those of PostgreSQL 17.2.
- **Scope: SELECT** only under **READ COMMITTED**; DML/DDL and other isolation levels follow stock.
- **On-disk format / protocol:** unchanged (it is PostgreSQL 17.2); a stock cluster and MontuDB 4.8 are interchangeable on the same datadir.